

Hands-On Sensors Workshop

DHT11 Temperature Sensor → HC-SR04 Ultrasonic Sensor → Servo Motor + LCD

Yantrika: Robotics & Coding Club

RSA, IACS

September 28, 2026

What We Will Build

- 1 Understand what a sensor does
- 2 Read temperature and humidity using **DHT11**
- 3 Display the readings on a **16×2 I2C LCD**
- 4 Measure distance using **HC-SR04**
- 5 Display distance on the LCD
- 6 Control an **SG90 servo motor**
- 7 Combine everything into a simple ultrasonic scanning system

Workshop Philosophy

Build → Observe → Understand → Modify

Link to GitHub repository for workshop



<https://github.com/Yantrika-IACS/Sessions/tree/main/Workshops/workshop1.0>

What Is a Sensor?

A sensor converts a physical quantity into information that a microcontroller can process.

Sensor	Measures
DHT11	Temperature and humidity
HC-SR04	Distance using ultrasound
LDR	Light intensity
MPU6050	Acceleration and angular velocity

Today we will connect sensors to an Arduino and use an LCD to make their output visible.

Part I: DHT11 Temperature & Humidity

DHT11 provides:

- Temperature
- Relative humidity
- Digital output

First Goal

Read the sensor and display:

T = temperature, H = humidity

on the LCD.

How Does the DHT11 Know the Temperature?

Step 1 — Sense

- Temperature changes the electrical properties of the sensing element.
- The DHT11 detects this change.

Step 2 — Humidity

- Water vapour changes the capacitance of the humidity sensor.
- More/less capacitance → different humidity reading.

Step 3 — Process

- A tiny IC inside the DHT11 reads the sensors.
- It converts the measurements into digital data.

Step 4 — Communicate

- The DHT11 sends the result through its DATA pin.
- Arduino receives the digital information.

Physical World → Sensing → DHT11 IC → Digital Data → Arduino → LCD

DHT11 Wiring – Arduino Uno

DHT11	Arduino Uno
VCC	5V
GND	GND
DATA	D 12

Note

If you are using a bare DHT11 sensor rather than a module, the DATA line normally requires a pull-up resistor.

LCD: 16×2 I2C

The LCD lets us display the sensor output without relying on the Serial Monitor.

LCD I2C Pin	Arduino Uno
VCC	5V
GND	GND
SDA	A4
SCL	A5

Common I2C addresses are **0x27** and **0x3F**.

If the LCD is blank

Check the I2C address and adjust the contrast potentiometer on the LCD backpack.

DHT11 + LCD: First Program

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define DHTPIN 12
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27,16,2);

void setup() {
    dht.begin();
    lcd.init();
    lcd.backlight();

    lcd.setCursor(0,0);
    lcd.print("DHT11 Sensor");
    delay(2000);
}
```

DHT11 + LCD: Reading Values

```
void loop() {  
  float temperature = dht.readTemperature();  
  float humidity = dht.readHumidity();  
  if (isnan(temperature) || isnan(humidity)) {  
    lcd.clear();  
    lcd.setCursor(0,0);  
    lcd.print("Sensor Error");  
    delay(2000);  
    return;  
  }  
  lcd.clear();  
  lcd.setCursor(0,0);  
  lcd.print("Temp: ");  
  lcd.print(temperature,1);  
  lcd.print((char)223);  
  lcd.print("C");  
  lcd.setCursor(0,1);  
  lcd.print("Humidity: ");  
  lcd.print(humidity,1);  
  lcd.print("%");  
  delay(2000);  
}
```

What Is Happening in the Code?

- 1 `dht.begin()` starts communication with the DHT11.
- 2 `readTemperature()` obtains temperature.
- 3 `readHumidity()` obtains relative humidity.
- 4 `lcd.setCursor()` selects the LCD position.
- 5 `lcd.print()` writes text/numbers to the display.
- 6 `delay(2000)` waits before taking the next reading.

Mini Challenge

Change the LCD layout so that the first line shows temperature and the second line shows humidity with your own labels.

Part II: HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance using a pulse of ultrasound.

$$f \approx 40 \text{ kHz}$$

Basic sequence:

Trigger → Ultrasonic pulse → Reflection → Echo

The Arduino measures how long the echo takes to return.

HC-SR04 Wiring

HC-SR04	Arduino Uno
VCC	5V
GND	GND
TRIG	D5
ECHO	D6

Distance from Time of Flight

Let the measured echo time be Δt .

The sound travels:

Sensor $\rightarrow$ Object $\rightarrow$ Sensor

Therefore the measured path is twice the actual distance:

$$2d = v\Delta t$$

Hence:

$$d = \frac{v\Delta t}{2}$$

For air:

$$v \approx 343 \text{ m/s}$$

Why Do We Divide by 2?

Suppose the object is at distance d .

$$\underbrace{d}_{\text{outgoing}} + \underbrace{d}_{\text{return}} = 2d$$

The HC-SR04 measures the total travel time of both paths.
Therefore:

$$d = \frac{\text{total travel distance}}{2}$$

This is the key idea behind ultrasonic ranging.

Workshop Challenge – Distance

Try the following:

- ① Put your hand in front of the sensor.
- ② Move it closer and farther away.
- ③ Observe the LCD reading.
- ④ Compare the measured distance with a ruler.

Question

Why does the measured distance change when the object moves?

Part III: SG90 Servo Motor

A servo converts a control signal into an angular position.

Typical hobby servo:

$$0^{\circ} \rightarrow 180^{\circ}$$

The Arduino controls the servo using a periodic control signal.

Important

The exact pulse width corresponding to an angle depends on the particular servo. The commonly used values are approximate.

Servo Wiring – SG90

SG90 Wire	Arduino
Brown/Black	GND
Red	5V
Orange/Yellow	D9

Power

For larger servos or multiple servos, use an appropriate external supply and connect the grounds together.

Now combine:

Servo + HC-SR04 + LCD

At every angle:

- 1 Rotate servo to angle θ
- 2 Measure distance r
- 3 Display θ and r on LCD

The LCD can show:

Angle: 90

Dist: 35.4 cm

Combined Wiring

Component	Arduino Uno Pin
HC-SR04 TRIG	D5
HC-SR04 ECHO	D6
SG90 Signal	D9
LCD SDA	A4
LCD SCL	A5

All components should share a common GND.

Ultrasonic Radar: Hardware Setup

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

#define TRIG 5
#define ECHO 6
#define SERVO_PIN 9

Servo s;
LiquidCrystal_I2C lcd(0x27,16,2);

void setup() {
  pinMode(TRIG,OUTPUT);
  pinMode(ECHO,INPUT);

  s.attach(SERVO_PIN);
  Serial.begin(9600);

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0,0);
  lcd.print("Radar Starting");
  delay(1000);
  lcd.clear();
}
```

Ultrasonic Radar: Measuring Distance

```
long getDistance() {  
  
    digitalWrite(TRIG,LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(TRIG,HIGH);  
    delayMicroseconds(10);  
  
    digitalWrite(TRIG,LOW);  
  
    long duration = pulseIn(ECHO,HIGH,30000);  
  
    if (duration == 0)  
        return 400;  
  
    return duration * 0.0343 / 2;  
}
```

Ultrasonic Radar: Scanning — Part 2

```
void loop() {  
  
    // 0 -> 180  
    for (int angle=0; angle<=180; angle+=2) {  
  
        s.write(angle);  
        delay(20);  
  
        long distance = getDistance();  
  
        // Send angle and distance to Python  
        Serial.print(angle);  
        Serial.print(",");  
        Serial.println(distance);  
  
        // Display on LCD  
        lcd.clear();  
  
        lcd.setCursor(0,0);  
        lcd.print("Angle: ");  
        lcd.print(angle);  
        lcd.print((char)223);  
  
        lcd.setCursor(0,1);  
        lcd.print("Dist: ");  
        lcd.print(distance);  
        lcd.print(" cm");  
  
        delay(50);  
    }  
}
```

Ultrasonic Radar: Scanning — Part 3

```
// 180 -> 0
for (int angle=180; angle>=0; angle-=2) {

    s.write(angle);
    delay(20);

    long distance = getDistance();

    // Send data to Python
    Serial.print(angle);
    Serial.print(",");
    Serial.println(distance);

    // LCD output
    lcd.clear();

    lcd.setCursor(0,0);
    lcd.print("Angle: ");
    lcd.print(angle);
    lcd.print((char)223);

    lcd.setCursor(0,1);
    lcd.print("Dist: ");
    lcd.print(distance);
    lcd.print(" cm");

    delay(50);
}
```

From Polar to Cartesian Coordinates

The scanner measures:

$$(r, \theta)$$

Convert to Cartesian coordinates:

$$x = r \cos \theta$$

$$y = r \sin \theta$$

So each ultrasonic measurement can become a point in a 2D map.

This is the basic mathematical idea behind a simple ultrasonic radar.

Final Workshop Challenge

Build a working system that:

- ① Measures temperature and humidity with DHT11.
- ② Measures distance with HC-SR04.
- ③ Rotates the sensor using the SG90.
- ④ Displays useful information on the I2C LCD.

Bonus

Change the LCD message depending on distance:

- Near object → "OBJECT NEAR"
- Far object → "CLEAR"

Key Takeaways

- Sensors convert physical quantities into usable data.
- DHT11 gives temperature and humidity.
- HC-SR04 measures distance using time of flight.
- A servo provides controlled angular motion.
- I2C LCD provides a simple local display.
- Combining sensors + actuation creates the foundation of robotic systems.

Build it. Measure it. Understand it.

Thank You!

Yantrika: Robotics & Coding Club
RSA, IACS